

GKR Protocol. Offline Memory Checking

July 17, 2025

Distributed Lab

 zkdl-camp.github.io

 github.com/ZKDL-Camp



Introduction

Recap: Multivariate World and Sum-Check



Univariate World:

$$p(X) = q(X) \prod_{u \in \Omega} (X - u)$$

Multivariate World:

$$\sum_{\mathbf{b} \in \{0,1\}^\ell} f(b_1, \dots, b_\ell) = H$$

Goal: build the set of constraints that boil down to **Sum-Check**:

$$\sum_{(b_1, \dots, b_\ell) \in \{0,1\}^\ell} f(b_1, \dots, b_\ell) = H$$

Cost: Quasilinear prover, logarithmic verifier and proof size.

GKR Protocol

Motivation

Goal: Build sumcheck-based version of the circuit arithmetization.

Suppose we are given the **layered** fan-in two arithmetical circuit $C : \mathbb{F}^n \rightarrow \mathbb{F}^m$ of size S (number of gates). The *layered* here means that the circuit C can be decomposed into d layers (note that GKR can be generalized to the unstructured arithmetical circuits as well).

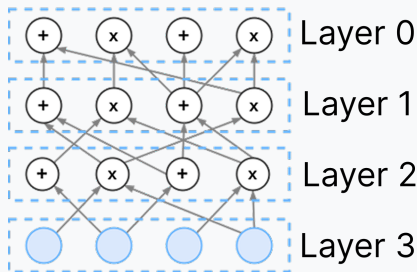


Figure: Layered Circuit Structure of $d = 4$ layers.

Spoilers on Performance

The GKR protocol allows to achieve the following performance:

- The communication consists of $O(d \cdot \text{polylog}(S))$ field elements.
- The verifier runs in $O(n + d \cdot \text{polylog}(S))$ time.
- The prover runs in $O(\text{poly}(S))$ time.
- The soundness error is just $O(d \log(S) / |\mathbb{F}|)$.

Assumptions:

- Assume we have d rounds in total. Output layer is the 0th layer.
- Each layer consists of S_i gates.
- Assume $S_i = 2^{v_i}$ is the power of two

Concrete Circuit

Layer 3 (Inputs)

$$S_3 = 8, v_3 = 3$$

Layer 1

$$S_1 = 4, v_1 = 2$$

Layer 2

$$S_2 = 4, v_2 = 2$$

Layer 0 (Output)

$$S_0 = 2, v_0 = 1$$

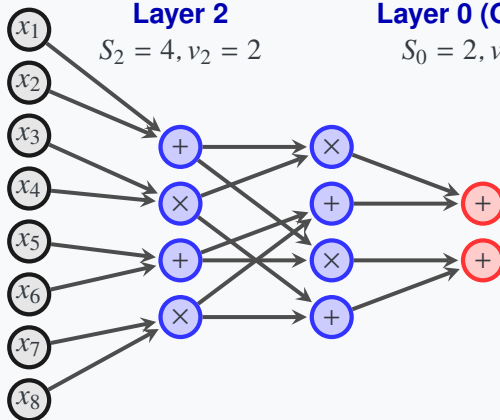
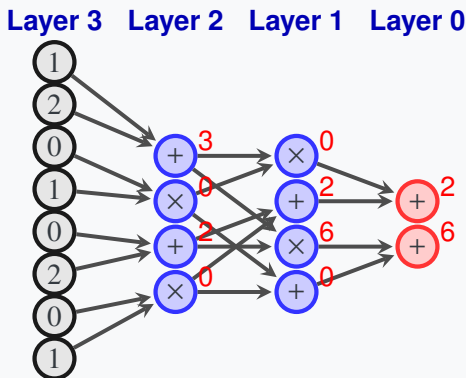


Figure: Example layered arithmetical circuit $C : \mathbb{F}^8 \rightarrow \mathbb{F}^2$ with $d = 3$ layers.

Gates Encoding

Gates Encoding. Suppose $W_i : \{0, 1\}^{v_i} \rightarrow \mathbb{F}$ is structured so that it outputs the value of the i -th layer gate given the gate label. Assume MLE of W_i is $\widetilde{W}_i : \mathbb{F}^{v_i} \rightarrow \mathbb{F}$.

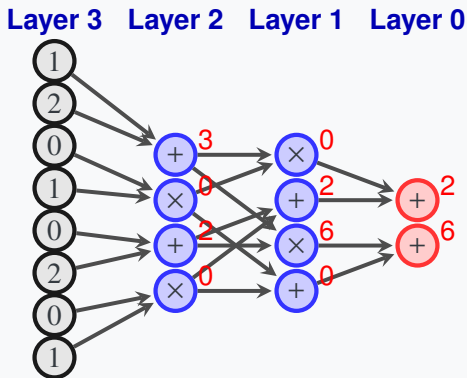


$$W_1(0, 0) = 0, \quad W_1(0, 1) = 2, \quad W_1(1, 0) = 6, \quad W_1(1, 1) = 0$$

MLE Extension: $\widetilde{W}_1(X_1, X_2) = 2(1 - X_1)X_2 + 6X_1(1 - X_2)$

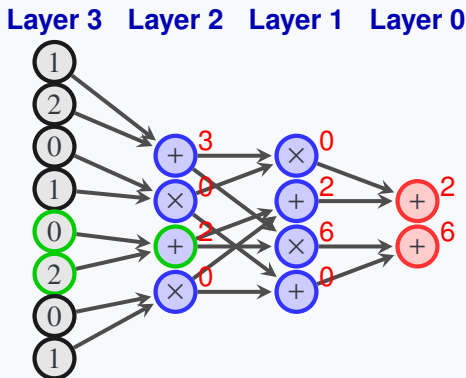
Wiring Encoding

Wiring Predicates. $\text{in}_{1,i}, \text{in}_{2,i} : \{0, 1\}^{v_i} \rightarrow \{0, 1\}^{v_{i+1}}$ indicate which pairs of wiring are connected to the i^{th} layer gate from the layer $i + 1$.



Wiring Encoding

Wiring Predicates. $\text{in}_{1,i}, \text{in}_{2,i} : \{0, 1\}^{v_i} \rightarrow \{0, 1\}^{v_{i+1}}$ indicate which pairs of wiring are connected to the i^{th} layer gate from the layer $i + 1$.



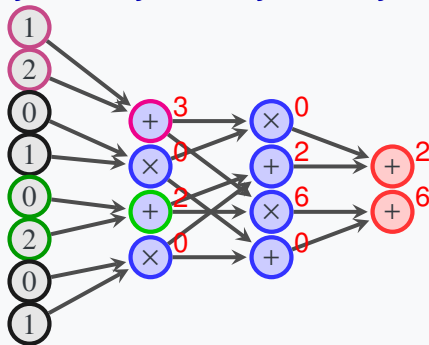
$$\text{in}_{1,2}(1,0) = (1,0,1), \quad \text{in}_{2,2}(1,0) = (1,1,0).$$

Operations Encoding

Operations Encodings. $\text{add}, \text{mul} : \{0, 1\}^{v_i+2v_{i+1}} \rightarrow \{0, 1\}$:

$\text{add}(a, b, c) = 1 \iff (b, c) = (\text{in}_{1,i}(a), \text{in}_{2,i}(a))$ and a is addition gate

Layer 3 Layer 2 Layer 1 Layer 0



add_2 is non-zero : $((0, 0), (0, 0, 0), (0, 0, 1)), ((1, 0), (1, 0, 0), (1, 0, 1)).$

$$\widetilde{\text{add}}_2(\mathbf{X}, \mathbf{Y}, \mathbf{Z}) = (1 - X_1)(1 - X_2)(1 - Y_1)(1 - Y_2)(1 - Y_3)(1 - Z_1)(1 - Z_2)Z_3$$

Reducing to Sum-Check

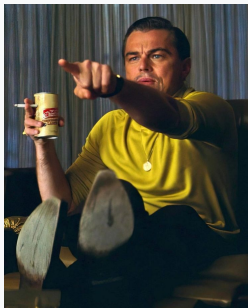
Remark

Note that the operations encodings add_i and mul_i (and thus MLEs $\widetilde{\text{add}}_i$ and $\widetilde{\text{mul}}_i$) do not depend on the solution witness $\{\mathbf{x}^{(i)}\}_{i \in [d+1]}$, while the gates encodings W_i do depend.

Idea: Prover sends the claimed value of $\widetilde{W}_0$ (say, $D : \{0, 1\}^{v_0} \rightarrow \mathbb{F}$), then reduce the claim to the next around with $\widetilde{W}_1$ (in general, prove the reducing from $\widetilde{W}_i$ to $\widetilde{W}_{i+1}$).



Sum-Check Protocol Applied



Lemma

The following statement holds:

$$\begin{aligned}\widetilde{W}_i(\mathbf{z}) = \sum_{b,c \in \{0,1\}^{v_{i+1}}} & \left[\widetilde{\text{add}}_i(\mathbf{z}, a, b)(\widetilde{W}_{i+1}(b) + \widetilde{W}_{i+1}(c)) \right. \\ & \left. + \widetilde{\text{mul}}_i(\mathbf{z}, b, c)\widetilde{W}_{i+1}(b)\widetilde{W}_{i+1}(c) \right].\end{aligned}$$

Why Lemma works?

Both sides are multilinear polynomials, thus it suffices to check the equality only over the boolean hypercube $\mathbf{z} \in \{0, 1\}^{v_i}$.

Fix $\mathbf{z} = \mathbf{z}_0 \in \{0, 1\}^{v_i}$. Without loss of generality, assume $\mathbf{z}_0$ is the addition gate. This way, we reduced the check to:

$$\widetilde{W}_i(\mathbf{z}_0) = \sum_{\mathbf{b}, \mathbf{c} \in \{0, 1\}^{v_{i+1}}} \widetilde{\text{add}}_i(\mathbf{z}, \mathbf{a}, \mathbf{b})(\widetilde{W}_{i+1}(\mathbf{b}) + \widetilde{W}_{i+1}(\mathbf{c}))$$

According to $\widetilde{\text{add}}$ definition, the only term that is not zero in the sum is for $(\mathbf{b}, \mathbf{c}) = (\text{in}_{1,i}(\mathbf{z}_0), \text{in}_{2,i}(\mathbf{z}_0))$. Therefore, our sum is:

$$\widetilde{W}_i(\mathbf{z}_0) = \widetilde{W}_{i+1}(\text{in}_{1,i}(\mathbf{z}_0)) + \widetilde{W}_{i+1}(\text{in}_{2,i}(\mathbf{z}_0))$$

Key Procedure

Apply the Sum-Check protocol on the function

$$f_i(\mathbf{b}, \mathbf{c}; \mathbf{r}_i) = \widetilde{\text{add}}_i(\mathbf{r}_i, \mathbf{b}, \mathbf{c})(\widetilde{W}_{i+1}(\mathbf{b}) + \widetilde{W}_{i+1}(\mathbf{c})) + \widetilde{\text{mul}}_i(\mathbf{r}_i, \mathbf{b}, \mathbf{c})\widetilde{W}_{i+1}(\mathbf{b})\widetilde{W}_{i+1}(\mathbf{c}).$$

Caveat with Sum-Check

Note that the verifier $\mathcal{V}$ does not know $\widetilde{W}_{i+1}$.

In fact, he does not need to until the last round, where he needs to call an oracle access O^{f_i} at $(\mathbf{b}^*, \mathbf{c}^*) \leftarrow \$ \mathbb{F}^{2v_{i+1}}$.

This requires evaluating:

- $\widetilde{\text{add}}_i(\mathbf{r}_i, \mathbf{b}^*, \mathbf{c}^*)$ — can be done by $\mathcal{V}$.
- $\widetilde{\text{mul}}_i(\mathbf{r}_i, \mathbf{b}^*, \mathbf{c}^*)$ — can be done by $\mathcal{V}$.
- $\widetilde{W}_{i+1}(\mathbf{b}^*)$ and $\widetilde{W}_{i+1}(\mathbf{c}^*)$ — $\mathcal{V}$ needs $\mathcal{P}$'s assistance.

$\mathcal{P}$ sends two values $z_b = \widetilde{W}_{i+1}(\mathbf{b}^*)$ and $z_c = \widetilde{W}_{i+1}(\mathbf{c}^*)$. If we had only one value to check, we could use the standard Sum-Check reduction, but here we have two randomnesses!

Line Restriction Trick

Proposition

Let $\ell : \mathbb{F} \rightarrow \mathbb{F}^{v_{i+1}}$ be the line such that $\ell(0) = \mathbf{b}^*$ and $\ell(1) = \mathbf{c}^*$. Then, the prover $\mathcal{P}$ sends the univariate polynomial $q(X)$ claimed to be equal to $\widetilde{W}_{i+1} \circ \ell$ — the restriction of $\widetilde{W}_{i+1}$ to the line ℓ . $\mathcal{V}$ checks whether indeed $\ell(0) = \mathbf{z}_b$ and $\ell(1) = \mathbf{z}_c$, then chooses a random point $\mathbf{r}^* \xleftarrow{R} \mathbb{F}^{v_{i+1}}$ and checks whether $\widetilde{W}_{i+1}(\ell(\mathbf{r}^*)) = q(\mathbf{r}^*)$.

This way, the interaction ends with new claim about next(previous) layer $\widetilde{W}_{i+1}(\mathbf{r}_{i+1})$ with $\mathbf{r}_{i+1} = \ell(\mathbf{r}^*)$.

In the last round, $\mathcal{V}$ computes $\widetilde{W}_d(\mathbf{r}_d)$ on his own.

Protocol Summary

- $\mathcal{P}$ sends function $D : \{0, 1\}^{v_0} \rightarrow \mathbb{F}$, claimed to equal W_0 .
- $\mathcal{V}$ picks random $\mathbf{r}_0 \leftarrow \$ \mathbb{F}^{v_0}$ and lets $m_0 \leftarrow \widetilde{D}(\mathbf{r}_0)$.
- For each round $i \in [d]$ do the following:
 - Define the $2v_{i+1}$ -variate polynomial:

$$f_i(\mathbf{b}, \mathbf{c}; \mathbf{r}_i) = \widetilde{\text{add}}_i(\mathbf{r}_i, \mathbf{b}, \mathbf{c})(\widetilde{W}_{i+1}(\mathbf{b}) + \widetilde{W}_{i+1}(\mathbf{c})) + \widetilde{\text{mul}}_i(\mathbf{r}_i, \mathbf{b}, \mathbf{c})\widetilde{W}_{i+1}(\mathbf{b})\widetilde{W}_{i+1}(\mathbf{c}).$$
 - $\mathcal{P}$ claims $\sum_{\mathbf{b}, \mathbf{c} \in \{0, 1\}^{v_{i+1}}} f_i(\mathbf{b}, \mathbf{c}; \mathbf{r}_i) = m_i$.
 - $\mathcal{P}$ and $\mathcal{V}$ interact using Sum-Check protocol until the last round when $\mathcal{V}$ needs to evaluate f_i at $\mathbf{b}^*, \mathbf{c}^* \leftarrow \$ \mathbb{F}^{v_{i+1}}$.
 - $\mathcal{P}$ and $\mathcal{V}$ compute the line $\ell : \mathbb{F} \rightarrow \mathbb{F}^{v_{i+1}}$ s.t. $\ell(0) = \mathbf{b}^*$ and $\ell(1) = \mathbf{c}^*$.
 - $\mathcal{P}$ sends q claimed to equal $\widetilde{W}_{i+1} \circ \ell$.
 - $\mathcal{V}$ validates the last round of sum-check using $\ell(0)$ and $\ell(1)$, then chooses $\mathbf{r}^* \leftarrow \$ \mathbb{F}^{v_{i+1}}$ and sets $\mathbf{r}_{i+1} \leftarrow \ell(\mathbf{r}^*)$ and $m_{i+1} \leftarrow q(\mathbf{r}_{i+1})$.
 - The check reduces to verifying $\widetilde{W}_{i+1}(\mathbf{r}_{i+1}) = m_{i+1}$.
- $\mathcal{V}$ directly checks whether $m_d = \widetilde{W}_d(\mathbf{r}_d)$.

Thank you for your attention



 zkdl-camp.github.io
 github.com/ZKDL-Camp

